

CIND820 Big Data Analytics Project

Initial Results

Aaron Hosein

July 14, 2025

The following dataset was retrieved from Kaggle under a public domain license.

The dataset contains information on new vehicles released during 2024. The below outlines a Machine Learning model comparison to predict the price of a vehicle given vehicle features. The three models compared include: linear regression, random forest and XGBoost.

```
In [5]: import pandas as pd
import matplotlib.pyplot as plt

# Load the CSV file
vehicle_df = pd.read_csv('vehicles_dataset.csv')

# Display the first few rows
vehicle_df.head()
```

Out[5]:

	name	description	make	model	type	year	price	engine	cylinders	fuel	mileage	transmission	trim	body
0	2024 Jeep Wagoneer Series II	\n \n Heated Leather Seats, Nav Sy...	Jeep	Wagoneer	New	2024	74600.0	24V GDI DOHC Twin Turbo	6.0	Gasoline	10.0	8-Speed Automatic	Series II	SUV
1	2024 Jeep Grand Cherokee Laredo	AI West is committed to offering every custome...	Jeep	Grand Cherokee	New	2024	50170.0	OHV	6.0	Gasoline	1.0	8-Speed Automatic	Laredo	SUV
2	2024 GMC Yukon XL Denali	NaN	GMC	Yukon XL	New	2024	96410.0	6.2L V-8 gasoline direct injection, variable V...	8.0	Gasoline	0.0	Automatic	Denali	SUV
3	2023 Dodge Durango Pursuit	White Knuckle Clearcoat 2023 Dodge Durango Pur...	Dodge	Durango	New	2023	46835.0	16V MPFI OHV	8.0	Gasoline	32.0	8-Speed Automatic	Pursuit	SUV
4	2024 RAM 3500 Laramie	\n \n 2024 Ram 3500 Laramie Billet...	RAM	3500	New	2024	81663.0	24V DDI OHV Turbo Diesel	6.0	Diesel	10.0	6-Speed Automatic	Laramie	Pickup Truck

```

In [6]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load the dataset
df_clean = pd.read_csv("vehicles_dataset.csv")

# Remove rows with missing values
df_clean = df_clean.dropna()

# Prepare dataframe by dropping redundant or irrelevant features and select target parameter
X = df_clean.drop(['price', 'name', 'description', 'type', 'engine', 'trim'], axis=1)
y = df_clean['price']

# Convert continuous price to categories for classification. pd.qcut to split continuous data into four equal
price_bins = pd.qcut(y, q=4, labels=False)
y = price_bins

# Identify categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()

# One-Hot encoding for categorical variables
preprocessor = ColumnTransformer(transformers=[('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols,
                                                remainder='passthrough')

# Train-test split data into 80-20 ratios
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Define machine learning models for comparison
models = {
    # Linear regression normally for continuous values, but used for classification by round continuous values
    'Linear Regression': LinearRegression(),

    # Classification model using 42 as a starting seed
    'Random Forest': RandomForestClassifier(random_state=42),

```

```

# Advanced classification model similar to random forest
# mlogloss selected for multi-class classification
# Uses 42 as a starting seed for reproducing model
# 50 estimators to control how complex the model computes
# max depth of 3 to control over-fitting but also complexity
'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='mlogloss', random_state=42, n_estimators=50)
}

# Store results for comparison
results = {}

# Cross-validation setup - compare model with multiple subsets of dataset to provide more reliable estimate
# StratifiedKFold specifically ensures each fold is proportional to remove bias
# Split the dataset into 5 folds
# Shuffle the data before splitting the folds
# 42 is random seed for reproducing results
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for model_name, model in models.items():
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('model', model)])
    pipeline.fit(X_train, y_train)

    # Train on test set
    if model_name == 'Linear Regression':
        y_pred_continuous = pipeline.predict(X_test)
        y_pred = np.clip(np.round(y_pred_continuous), 0, 3).astype(int)
    else:
        y_pred = pipeline.predict(X_test)

    # Accuracy and Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    acc = accuracy_score(y_test, y_pred)

    # Cross-validation scoring
    if model_name == 'Linear Regression':
        # For cross-validation, map continuous predictions to class labels
        def cv_scorer(pipe, X, y):
            y_pred_cv = pipe.predict(X)
            y_pred_cv = np.clip(np.round(y_pred_cv), 0, 3).astype(int)
            return accuracy_score(y, y_pred_cv)

        cv_scores = []

```

```

    for train_idx, val_idx in cv.split(X, y):
        pipe = Pipeline(steps=[('preprocessor', preprocessor), ('model', model)])
        pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
        cv_scores.append(cv_scorer(pipe, X.iloc[val_idx], y.iloc[val_idx]))
    else:
        cv_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')

results[model_name] = {
    'confusion_matrix': cm,
    'accuracy': acc,
    'cv_mean_accuracy': np.mean(cv_scores),
    'cv_std': np.std(cv_scores)
}

# Plot confusion matrix
plt.figure()
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title(f'Confusion Matrix - {model_name}')
plt.colorbar()
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.xticks(np.arange(4), [f'Class {i}' for i in range(4)])
plt.yticks(np.arange(4), [f'Class {i}' for i in range(4)])
plt.show()

# Display results
for model_name, metrics in results.items():
    print(f"Model: {model_name}")
    print(f"Train-Test Accuracy: {metrics['accuracy']:.2f}")
    print(f"Cross-Validation Mean Accuracy: {metrics['cv_mean_accuracy']:.2f} (± {metrics['cv_std']:.2f})")
    print("Confusion Matrix:")
    print(metrics['confusion_matrix'])
    print("\n")

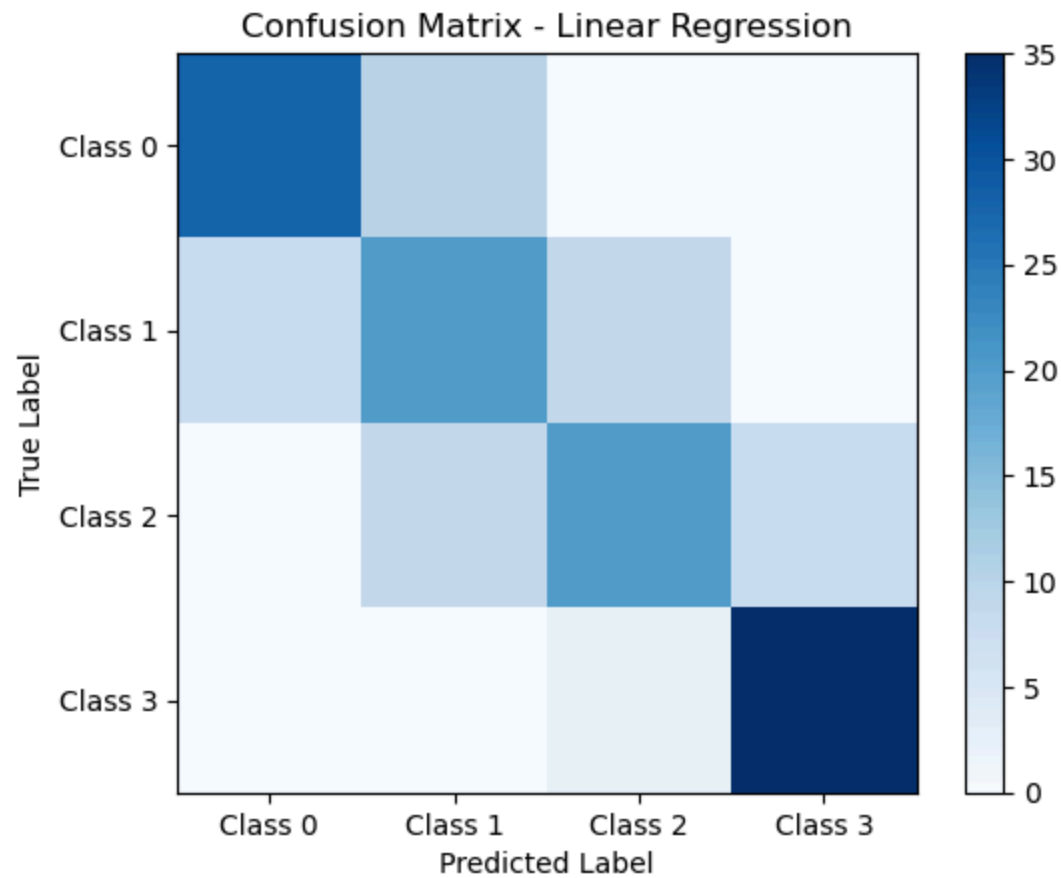
# Calculate the bin edges using pd.qcut
bins = pd.qcut(df_clean['price'], q=4, retbins=True)[1]
price_ranges = [(round(bins[i], 2), round(bins[i+1], 2)) for i in range(len(bins) - 1)]

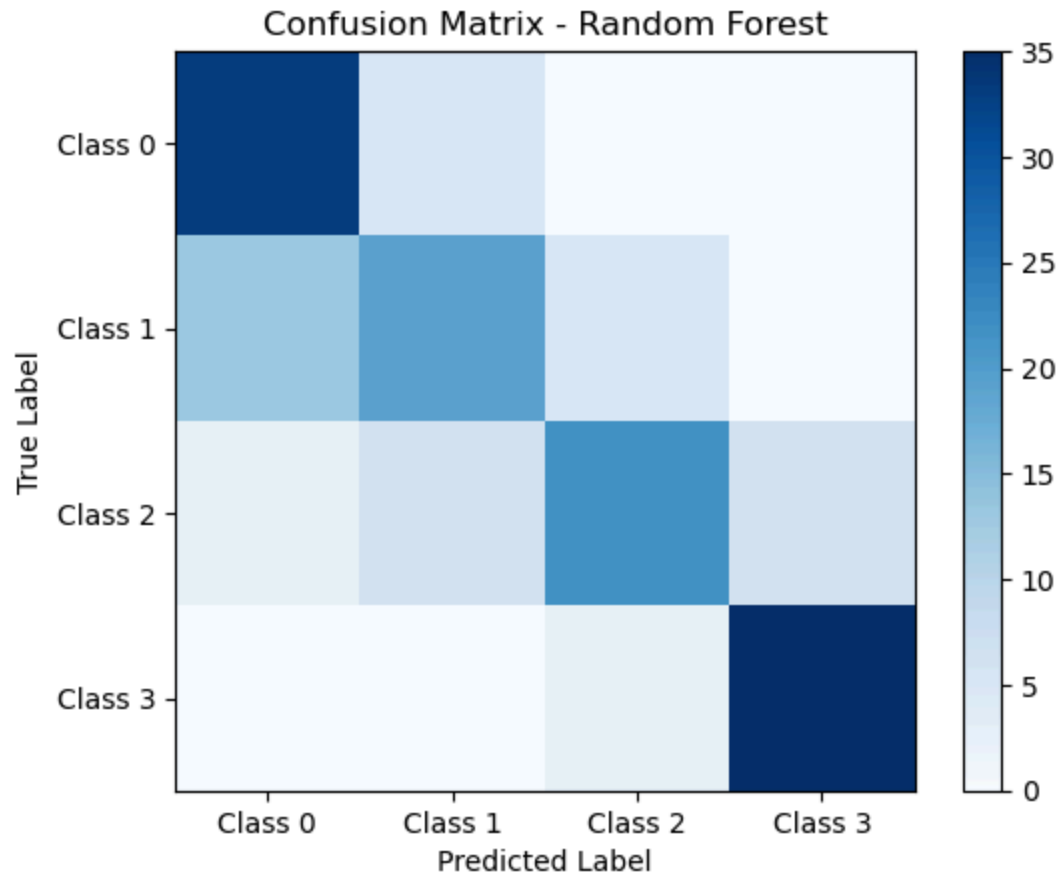
for idx, (low, high) in enumerate(price_ranges):
    print(f"Class {idx}: ${low} to ${high}")

```

C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use\_label\_encoder` is deprecated in 1.7.0.

warnings.warn("`use\_label\_encoder` is deprecated in 1.7.0.")





```
C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

```
C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

```
C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

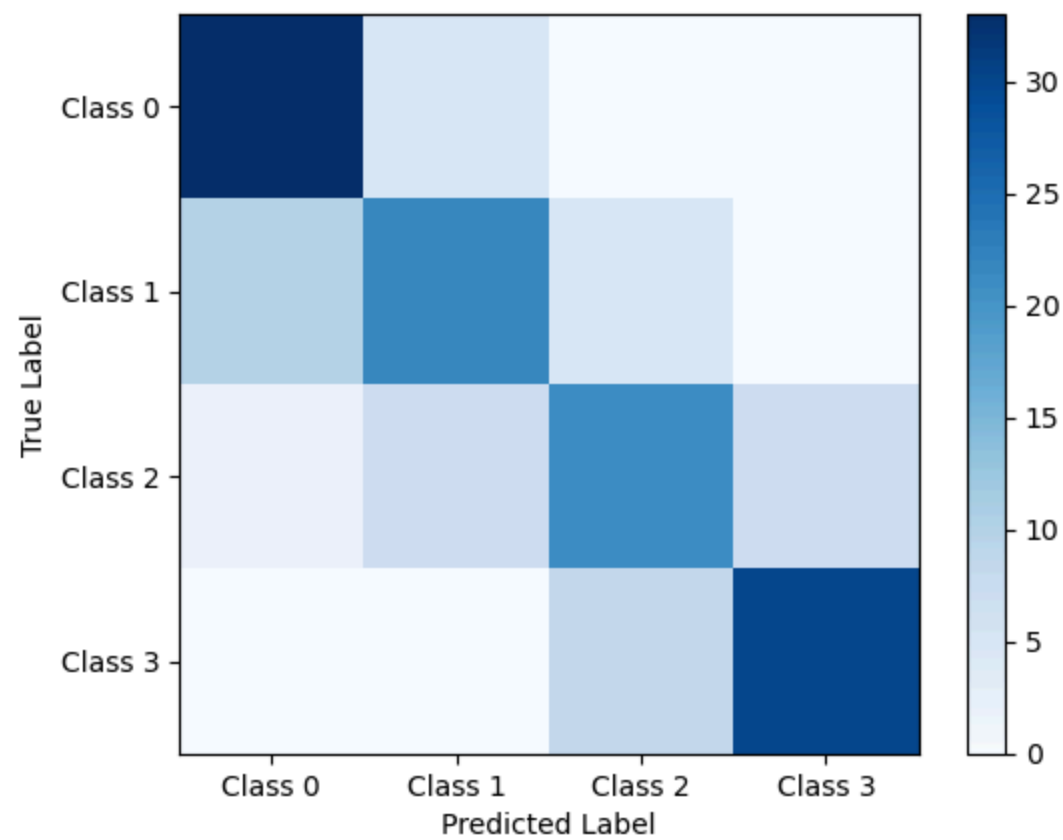
```
C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

```
C:\Users\aaahos\anaconda3\lib\site-packages\xgboost\sklearn.py:1395: UserWarning: `use_label_encoder` is deprecated in 1.7.0.
```

```
warnings.warn("`use_label_encoder` is deprecated in 1.7.0.")
```

Confusion Matrix - XGBoost




```
Model: Linear Regression
Train-Test Accuracy: 0.69
Cross-Validation Mean Accuracy: 0.72 ( $\pm$  0.02)
Confusion Matrix:
[[28 10  0  0]
 [ 8 20  9  0]
 [ 0  9 20  8]
 [ 0  0  3 35]]
```

```
Model: Random Forest
Train-Test Accuracy: 0.73
Cross-Validation Mean Accuracy: 0.74 ( $\pm$  0.03)
Confusion Matrix:
[[33  5  0  0]
 [13 19  5  0]
 [ 3  6 22  6]
 [ 0  0  3 35]]
```

```
Model: XGBoost
Train-Test Accuracy: 0.71
Cross-Validation Mean Accuracy: 0.72 ( $\pm$  0.06)
Confusion Matrix:
[[33  5  0  0]
 [10 22  5  0]
 [ 2  7 21  7]
 [ 0  0  8 30]]
```

```
Class 0: $0.0 to $35786.5
Class 1: $35786.5 to $46862.5
Class 2: $46862.5 to $58895.5
Class 3: $58895.5 to $124250.0
```

In []: